

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS



Criptografía y Seguridad

TAREA 3

Profesor: José Antonio Macías García

Ayudante: Juan Angeles Hernández

Ayudante: Erick Bernal Márquez

Alumno: Gustavo Alonso Domínguez Sánchez

418047121

18 de Mayo de 2026

Problema 1: Predicados "Hard-Core" a partir de Funciones Unidireccionales (Parte 1)

Supongamos que el predicado $gl(x, r)$ no es hard-core. Es decir, existe un algoritmo probabilístico de tiempo polinomial A y un polinomio $p(\cdot)$ tal que:

$$\Pr_{x \leftarrow \{0,1\}^n} [A'(1^n, f(x)) \in f^{-1}(f(x))] \geq \frac{1}{4p(n)}$$

Sabemos por definición que f es una función unidireccional, lo que implica que es computacionalmente inviable invertirla. Entonces, el objetivo es construir un algoritmo probabilístico de tiempo polinomial A' a partir de A que logre invertir la función f con una probabilidad de al menos $1/4p(n)$. Pero como f es una función unidireccional por definición, esto generará una contradicción.

(1) Construcción del algoritmo A'

Para poder encontrar el valor de $x = x_1x_2...x_n$, donde cada x_i es un bit de x , el algoritmo A' recibe como entrada los parámetros 1^n y el valor $y = f(x)$. Su objetivo es aislar y deducir cada bit x_i de manera individual. De esta forma, nos apoyamos en las propiedades de la operación XOR ($\oplus$).

Definamos $e^i \in \{0,1\}^n$ como el i -ésimo vector de la base canónica, es decir, e^i es una cadena de n bits donde todos los bits son 0, excepto el bit en la posición i , que es 1. Por la definición del predicado gl , si evaluamos gl utilizando e^i como una variable aleatoria, obtenemos exactamente el i -ésimo bit de x :

$$gl(x, e^i) = \bigoplus_{j=1}^n (x_j \cdot e_j^i) = x_i \cdot 1 = x_i$$

Es claro que no se le puede simplemente a A por el valor de $g_l(x, e^i)$ y confiar en la respuesta, ya que por hipótesis el algoritmo A es probabilístico (con una relativa buena probabilidad de éxito) sobre un vector aleatorio r , sin embargo si le pasamos e^i , la probabilidad de éxito de A puede ser nula.

Entonces para el algoritmo A utilizaremos dos vectores que sí se distribuyen de manera uniformemente aleatoria:

- Generamos un vector $r \leftarrow \{0, 1\}^n$ al azar.
- Construimos un segundo vector calculando $r \oplus e^i$ (Obs: Dado que r es uniforme, al aplicarle un XOR con una constante e^i , el resultado $r \oplus e^i$ sigue siendo una cadena uniformemente distribuida en $\{0, 1\}^n$).

De esta forma, observemos qué sucede si sumamos (hacemos XOR) el predicado real de estos dos vectores es:

$$g_l(x, r) \oplus g_l(x, r \oplus e^i)$$

$$= g_l(x, r) \oplus (g_l(x, r) \oplus g_l(x, e^i)) \quad \text{por prop. distributiva}$$

$$= 0 \oplus g_l(x, e^i) = x_i \quad \text{dado que } a \oplus a = 0$$

$$\Rightarrow x_i = g_l(x, r) \oplus g_l(x, r \oplus e^i)$$

La expresión anterior asume que conocemos el valor verdadero de g_l , pero tenemos a A para obtenerlo. Por lo tanto y para fines prácticos:

$$x_i = b_1 \oplus b_2$$

donde:

- $b_1 = g_l(x, r) = A(f(x), r)$
- $b_2 = g_l(x, r \oplus e^i) = A(f(x), r \oplus e^i)$

Si A responde correctamente ambos términos, entonces esta conjetura será exactamente x_i . Como A es un algoritmo probabilístico y puede fallar, hacerlo una vez por bit es insuficiente. De esta forma, para amplificar nuestra probabilidad de éxito, nuestro algoritmo repetirá el proceso m veces, donde m será un polinomio dependiente de n y $p(n)$ como veremos más adelante.

Con todo lo anterior, el algoritmo probabilístico de tiempo polinomial $A'(1^n, y)$ es de la siguiente forma:

1. Fijamos un número de iteraciones m (polinomial).
2. Para cada i desde 1 hasta n (cada bit de x):
 - Iniciar un contador $\kappa_i = 0$
 - Repetir m veces:
 - Elegir $r \leftarrow \{0, 1\}^n$ uniformemente al azar.
 - Obtener $b_1 \leftarrow A(y, r)$
 - Obtener $b_2 \leftarrow A(y, r \oplus e^i)$
 - Si $b_1 \oplus b_2 = 1$, incrementar κ_i
 - Si $\kappa_i > \frac{m}{2}$, establecemos que el bit $x'_i = 1$, de lo contrario $x'_i = 0$.
3. Al finalizar el ciclo anterior, concatenamos los bits deducidos para formar $x' = x'_1 x'_2 \dots x'_n$.
4. Devolvemos x' .

Con esto se ha definido un algoritmo probabilístico A' que corre en tiempo polinomial (respecto a n).

(2) Probabilidad de éxito sobre x

Ahora nos interesa conectar la probabilidad del algoritmo de ataque A con la probabilidad de éxito del algoritmo A' para invertir la función f y recuperar la cadena completa x .

Nuestra hipótesis principal nos dice que la probabilidad de éxito de A , tomada sobre todas las combinaciones posibles de x y r , es relativamente alta:

$$\Pr_{x,r \leftarrow \{0,1\}^n} [A(f(x), r) = g(x, r)] \geq \frac{3}{4} + \frac{1}{p(n)}$$

Sin embargo, podría haber algunas cadenas x para las que A no es bueno y otras donde sí lo es. Definimos $s(x)$ como la probabilidad de éxito de A para una cadena x fija, eligiendo solo r al azar:

$$s(x) = \Pr_{r \leftarrow \{0,1\}^n} [A(f(x), r) = g(x, r)]$$

Clasificaremos una cadena x como "buena" si su probabilidad de éxito $s(x)$ es igual o mejor que la cota, es decir, el conjunto de cadenas buenas S_n se define como:

$$S_n = \left\{ x \in \{0,1\}^n \mid s(x) \geq \frac{3}{4} + \frac{1}{2p(n)} \right\}$$

La fracción de cadenas x que son "buenas" (i.e., la probabilidad de que al elegir una x al azar, caiga en S_n) es al menos:

$$\Pr [x \in S_n] \geq \frac{1}{2p(n)}$$

A partir de ahora asumamos un x arbitrario "bueno" ($x \in S_n$). En el algoritmo A' , para adivinar un bit x_i , consultamos a A dos veces: una con r y otra con $r \oplus e^i$ ($x_i = b_1 + b_2$). Para que esto de el valor correcto de x_i ambas consultas (b_1 y b_2) deben ser correctas o ambas incorrectas (porque el error se cancelaría con el XOR). Sin pérdida de generalidad, supondremos que ambas son correctas. Dado que x es "buena":

- La probabilidad de que b_1 sea incorrecto es a lo mucho $1 - \left(\frac{3}{4} + \frac{1}{2p(n)} \right) = \frac{1}{4} - \frac{1}{2p(n)}$

- La probabilidad de que b_2 sea incorrecto es exactamente la misma, ya que $r \oplus e^i$ también es una distribución uniformemente aleatoria.

La probabilidad de que al menos una de las dos consultas falle es menor o igual a la suma de sus probabilidades de fallo individuales:

$$\Pr[\text{Fallar en } b_1 \text{ o } b_2] \leq \left(\frac{1}{4} - \frac{1}{2\rho(n)}\right) + \left(\frac{1}{4} - \frac{1}{2\rho(n)}\right) = \frac{1}{2} - \frac{1}{\rho(n)}$$

Por lo tanto, la probabilidad de que ambas sean correctas (nuestro caso de éxito por cada iteración) es:

$$\Pr[\text{Éxito en una iteración}] \geq 1 - \left(\frac{1}{2} - \frac{1}{\rho(n)}\right) = \frac{1}{2} + \frac{1}{\rho(n)}$$

Ciertamente esta última probabilidad es mayor a $1/2$, pero sigue siendo riesgoso adivinar el bit en un solo intento. Por esta razón, el algoritmo A' repite el algoritmo m veces y toma la mayoría como resultado. Si repetimos experimentos independientes cuya probabilidad individual es mayor a $1/2$, la ley de los grandes números nos dice que la probabilidad de que la mayoría se equivoque irá disminuyendo cada vez más.

Sea $m = 2n \cdot \rho(n)^2$ el número polinomial de iteraciones, la probabilidad de fallar para un bit específico x_i se vuelve muy pequeña y acotada por arriba por $1/2n$.

Para que A' tenga éxito, necesita adivinar todos los n bits correctamente. Si la probabilidad de equivocarse en un solo bit es menor o igual a $1/2n$, entonces para los n bits la probabilidad será menor o igual a $n \cdot 1/2n = 1/2$.

Finalmente, la probabilidad total de que el algoritmo A' sea exitoso se calcula multiplicando la probabilidad de que se haya elegido una cadena "buena", por la probabilidad que A' logre descifrar esa cadena:

$$\begin{aligned} \Pr[A' \text{ exitoso}] &\geq \Pr[A' \text{ seleccionar } x \mid x \in S_n] \cdot \Pr[x \in S_n] \\ \Rightarrow \Pr[A' \text{ exitoso}] &\geq \left(\frac{1}{2}\right) \left(\frac{1}{2\rho(n)}\right) = \frac{1}{4\rho(n)} \end{aligned}$$

A partir de suponer que existe A , acabamos de demostrar que el algoritmo A' existe y que invierte la función f con una probabilidad no insignificante, pero por definición sabemos que f es invertible, llegando así a la contradicción. Entonces como A' no puede existir, entonces el algoritmo A tampoco puede.

Y si no existe ningún algoritmo de ataque A capaz de adivinar el bit $g_l(x, r)$ de manera segura, entonces **el bit es matemáticamente seguro y g_l es un predicado "hard-core"** ■

Problema 2: Predicados "Hard-Core" a partir de Funciones Unidireccionales (Parte 2)

Sabemos por hipótesis que el algoritmo A tiene una probabilidad de éxito significativa de adivinar el predicado $g_l(x, r)$. Se quiere demostrar que este éxito no es un simple producto de la suerte en unas cuantas cadenas x , sino que existe una cantidad significativa de cadenas x (el conjunto S_n) para las cuales el algoritmo de ataque A funciona eficazmente.

En especial se quiere probar que el tamaño de S_n es como mínimo, una fracción $1/2^p(n)$ del total de cadenas posibles (2^n).

A partir de la hipótesis del problema, la probabilidad se define sobre la elección uniforme de x y de r . Podemos reescribir esta probabilidad como el valor esperado de la probabilidad de éxito para cada x individual.

Tal como en el problema anterior, definamos $s(x)$ como la probabilidad de éxito de A cuando la cadena x está fija, y solo r varía al azar:

$$s(x) = \Pr_{r \leftarrow \{0,1\}^n} [A(f(x), r) = g_l(x, r)]$$

Dado que x se elige de manera uniforme dentro de un conjunto de 2^n cadenas posibles, la probabilidad global de éxito de A es el promedio de todos los $s(x)$:

$$\Pr_{\text{global}} = \Pr_{x, r \leftarrow \{0,1\}^n} [A(f(x), r) = g(\ell(x, r))] = \sum_{x \in \{0,1\}^n} \frac{1}{2^n} \cdot s(x)$$

Por nuestra hipótesis, sabemos que:

$$\Pr_{\text{global}} = \sum_{x \in \{0,1\}^n} \frac{1}{2^n} \cdot s(x) \geq \frac{3}{4} + \frac{1}{2p(n)} \quad (\text{I})$$

El problema 1 define el conjunto S_n (las cadenas "buenas") como:

$$S_n = \left\{ x \in \{0,1\}^n \mid s(x) \geq \frac{3}{4} + \frac{1}{2p(n)} \right\}$$

En consecuencia, todas las cadenas que no están en S_n (las cadenas "malas"), cumplen con la condición opuesta:

$$\forall x \notin S_n, s(x) < \frac{3}{4} + \frac{1}{2p(n)}$$

Definamos α como la fracción de cadenas que pertenecen a S_n . Es decir que $\alpha = |S_n|/2^n$.

Queremos demostrar que $|S_n| \geq \frac{1}{2p(n)} \cdot 2^n$, lo que es equivalente a demostrar que $\alpha \geq 1/2p(n)$. Por contradicción, supongamos lo contrario:

$$\alpha < \frac{1}{2p(n)} \quad (\text{II})$$

Tomamos la ecuación I y la dividimos en dos partes: los elementos que están en S_n y los que no lo están:

$$\Pr_{\text{global}} = \sum_{x \in \{0,1\}^n} \frac{1}{2^n} \cdot s(x) = \sum_{x \in S_n} \frac{1}{2^n} \cdot s(x) + \sum_{x \notin S_n} \frac{1}{2^n} \cdot s(x)$$

Ahora vamos a acotar esta expresión dándole un cada término su valor máximo posible para calcular el escenario más optimista.

- Para el grupo $x \in S_n$: La probabilidad $s(x)$ nunca puede ser mayor a 1. Así que sustituimos $s(x)$ por 1.
- Para el grupo $x \notin S_n$: Por definición, todos estos elementos tienen un $s(x)$ estrictamente menor a $\frac{3}{4} + \frac{1}{2\rho(n)}$. Sustituimos $s(x)$ por este límite superior.

Aplicando estas cotas superiores, obtenemos la siguiente desigualdad:

$$Pr_{\text{global}} < \left(\sum_{x \in S_n} \frac{1}{2^n} \cdot 1 \right) + \left(\sum_{x \notin S_n} \frac{1}{2^n} \cdot \left(\frac{3}{4} + \frac{1}{2\rho(n)} \right) \right)$$

Notemos que el primer término (suma) es exactamente α , y por lo tanto, la segunda suma tiene que ser $(1-\alpha)$. Sustituyendo:

$$Pr_{\text{global}} < \alpha \cdot 1 + (1-\alpha) \cdot \left(\frac{3}{4} + \frac{1}{2\rho(n)} \right)$$

Es claro que $(1-\alpha)$ siempre es menor a 1, por lo que podemos acotar aún más la expresión eliminando $(1-\alpha)$:

$$Pr_{\text{global}} < \alpha + 1 \cdot \left(\frac{3}{4} + \frac{1}{2\rho(n)} \right) = \alpha + \frac{3}{4} + \frac{1}{2\rho(n)}$$

Ahora introducimos la ecuación II que es la hipótesis de contradicción, por lo que la desigualdad queda como:

$$Pr_{\text{global}} < \left(\frac{1}{2\rho(n)} \right) + \frac{3}{4} + \frac{1}{2\rho(n)} = \frac{3}{4} + \frac{2}{2\rho(n)} = \frac{3}{4} + \frac{1}{\rho(n)}$$

Llegando así a la contradicción ya que por hipótesis del problema, la probabilidad global es mayor o igual al término desarrollado. Esto ocurrió como resultado de suponer que $\alpha < \frac{1}{2\rho(n)}$.

Por lo tanto, la afirmación contraria debe ser cierta: $\alpha \geq \frac{1}{2^{p(n)}}$.
 Y como $\alpha = \frac{|S_n|}{2^n}$, multiplicando ambos lados por 2^n , se concluye que:

$$|S_n| \geq \frac{1}{2^{p(n)}} \cdot 2^n \quad \blacksquare$$

Problema 3: Predicados "Hard-Core" a partir de Funciones Unidireccionales (Parte 3)

Queremos probar que, si tomamos una cadena x que pertenece al conjunto de cadenas buenas S_n , la probabilidad de que A adivine correctamente el bit para las consultas $b_1 = A(f(x), r)$ y $b_2 = A(f(x), r \oplus e_i)$ simultáneamente, esté acotada por abajo por $\frac{1}{2} + \frac{1}{p(n)}$.

Sea $x \in S_n$ una cadena fija. Nuestro espacio de probabilidad está definido únicamente sobre la elección aleatoria y uniforme de la cadena $r \leftarrow \{0, 1\}^n$.

Sean los siguientes dos eventos de probabilidad en este espacio:

- E_1 : El algoritmo A acierta la primera consulta b_1 .

$$E_1 = \{r \in \{0, 1\}^n \mid A(f(x), r) = g(x, r)\}$$

- E_2 : El algoritmo A acierta la segunda consulta b_2 .

$$E_2 = \{r \in \{0, 1\}^n \mid A(f(x), r \oplus e_i) = g(x, r \oplus e_i)\}$$

Dado que $x \in S_n$ por la definición del conjunto S_n , establecido en el problema 2, sabemos que la probabilidad de éxito de A sobre la relación r es alta, en particular:

$$\Pr[E_1] \geq \frac{3}{4} + \frac{1}{2^{p(n)}}$$

Para el evento E_2 notemos que r es una variable aleatoria distribuida uniformemente en $\{0,1\}^n$, entonces para cualquier constante c (como el vector e_i), la operación $r \oplus c$ también genera una distribución uniforme en $\{0,1\}^n$, por lo que la probabilidad de éxito de A sobre esta variable es idéntica a la de la primera consulta:

$$\Pr[E_2] \geq \frac{3}{4} + \frac{1}{2\rho(n)}$$

El problema pide calcular la probabilidad de que ambos eventos ocurran al mismo tiempo. Esto es la probabilidad de la intersección $\Pr[E_1 \cap E_2]$. Por definición, la probabilidad de la unión de dos eventos es:

$$\Pr[E_1 \cup E_2] = \Pr[E_1] + \Pr[E_2] - \Pr[E_1 \cap E_2]$$

$$\Rightarrow \Pr[E_1 \cap E_2] = \Pr[E_1] + \Pr[E_2] - \Pr[E_1 \cup E_2]$$

Por los axiomas de Kolmogorov, sabemos que la probabilidad de cualquier evento, incluyendo la unión $\Pr[E_1 \cup E_2]$, no puede exceder 1. Sustituimos esta cota superior en la ecuación para obtener la siguiente desigualdad (desigualdad de Boole):

$$\Pr[E_1 \cap E_2] \geq \Pr[E_1] + \Pr[E_2] - 1$$

Sustituimos las cotas inferiores individuales de E_1 y E_2 :

$$\Pr[E_1 \cap E_2] \geq \left(\frac{3}{4} + \frac{1}{2\rho(n)}\right) + \left(\frac{3}{4} + \frac{1}{2\rho(n)}\right) - 1$$

$$\Rightarrow \Pr[E_1 \cap E_2] \geq \frac{6}{4} + \frac{2}{2\rho(n)} - 1 = \frac{3}{2} + \frac{1}{\rho(n)} - 1$$

$$\Rightarrow \Pr[E_1 \cap E_2] \geq \frac{1}{2} + \frac{1}{\rho(n)} \blacksquare$$

Problema 4: Una clave y un secreto en AES

En este ejercicio se aplican técnicas de criptoanálisis para recuperar una clave secreta de 50 caracteres oculta en un texto con ruido. En particular, las acciones a realizar consisten en procesar un archivo de texto (`file.txt`) mediante reglas específicas basadas en números primos y desplazamientos alfabéticos para construir la clave. Posteriormente, esta clave debe validarse en un listado de hashes SHA-256 y utilizarse para abrir un mensaje cifrado mediante el algoritmo moderno AES-GCM (Advanced Encryption Standard - Galois/Counter Mode).

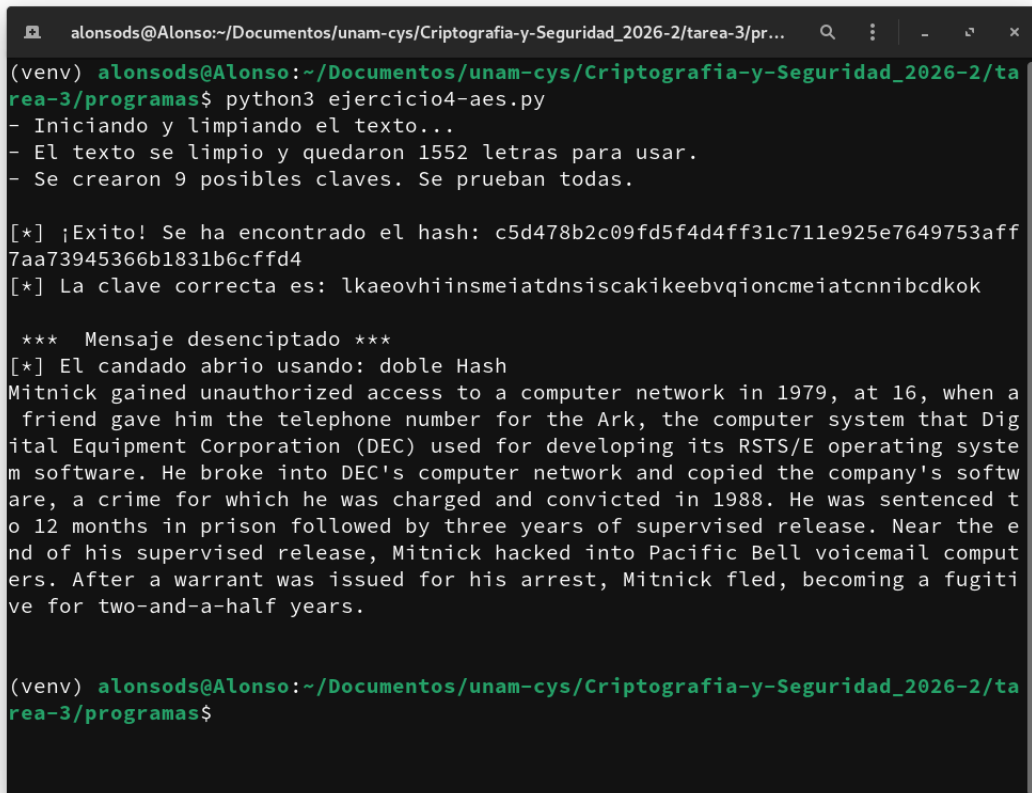
Para resolver este problema, el código se diseñó de manera modular en Python, dividiendo las tareas en estos bloques:

- Funciones básicas: Contiene las funciones para la generación de números primos y la limpieza y normalización de caracteres.
- Generación de claves: Para analizar más de un tipo de combinación, esta parte contiene la lógica de mezcla y desplazamiento para generar un conjunto selecto de claves, las cuales se considera que son las mejores candidatas a ser las correctas de acuerdo a las reglas dadas.
- Comprobación y descifrado: Verifica que la clave de nuestro conjunto anterior, es la correcta. Luego de hallarla (si existe), descifra el texto encriptado.
- Programa principal: Es el programa que contiene todo el flujo de autenticación:
 - Se lee el archivo `file.txt` y se transforma en una cadena continua de letras minúsculas, eliminando cualquier ruido que pueda desplazar los índices de los números primos.
 - Utilizando los primeros 50 números primos como índices (ajustados con un desplazamiento de $p - 2$), el programa extrae caracteres del texto limpio de tres formas distintas para cubrir posibles variantes del algoritmo.
 - Cada extracción se combina con la palabra base `kevinmitnick` mediante diferentes operaciones alfabéticas, resultando en un total de 9 variantes de la clave de 50 caracteres.
 - El programa calcula el SHA-256 de cada variante y lo compara con el archivo `hashes.txt`. Al encontrar una coincidencia, el programa detiene la búsqueda y marca esa clave como la ganadora.

- Finalmente, se procesa el archivo `cipher.txt` en formato hexadecimal. El programa extrae los 12 bytes correspondientes al nonce y utiliza la clave descubierta para descifrar el mensaje.

Ejecución del programa

```
$ python3 ejercicio4-aes.py
```



```
alonsods@Alonso:~/Documentos/unam-cys/Criptografia-y-Seguridad_2026-2/tarea-3/pr...  
(venv) alonsods@Alonso:~/Documentos/unam-cys/Criptografia-y-Seguridad_2026-2/tarea-3/programas$ python3 ejercicio4-aes.py  
- Iniciando y limpiando el texto...  
- El texto se limpio y quedaron 1552 letras para usar.  
- Se crearon 9 posibles claves. Se prueban todas.  
  
[*] ¡Exito! Se ha encontrado el hash: c5d478b2c09fd5f4d4ff31c711e925e7649753aff7aa73945366b1831b6cffd4  
[*] La clave correcta es: lkaeovhiinsmeiatdnsiscakikeebvqioncmeiatcnnibcdkok  
  
*** Mensaje descriptado ***  
[*] El candado abrio usando: doble Hash  
Mitnick gained unauthorized access to a computer network in 1979, at 16, when a friend gave him the telephone number for the Ark, the computer system that Digital Equipment Corporation (DEC) used for developing its RSTS/E operating system software. He broke into DEC's computer network and copied the company's software, a crime for which he was charged and convicted in 1988. He was sentenced to 12 months in prison followed by three years of supervised release. Near the end of his supervised release, Mitnick hacked into Pacific Bell voicemail computers. After a warrant was issued for his arrest, Mitnick fled, becoming a fugitive for two-and-a-half years.  
  
(venv) alonsods@Alonso:~/Documentos/unam-cys/Criptografia-y-Seguridad_2026-2/tarea-3/programas$
```

Figura 1: Ejecución del programa `ejercicio4-aes.py`

Problema 5: El gran secreto dentro de una imagen

El objetivo de este problema fue aplicar los principios esteganográficos mediante los comandados dados sobre el archivo de imagen `mao-shi.jpg` y descubrir una bandera (flag) secreta. Para la resolución de este problema, se utilizó la máquina virtual de Kali Linux y se siguieron los siguientes pasos:

1. Inicialmente, fue necesario instalar el software requerido: `steghide` y `jp2a`. `Steghide` es una herramienta clásica de línea de comandos que utiliza un algoritmo esteganográfico para incrustar o extraer datos de archivos multimedia (como JPEG o BMP), protegiéndolos adicionalmente con una contraseña. Por otro lado, `jp2a` es un conversor que permite transformar imágenes a arte ASCII.

```
$ sudo apt-get update  
$ sudo apt-get install steghide jp2a
```

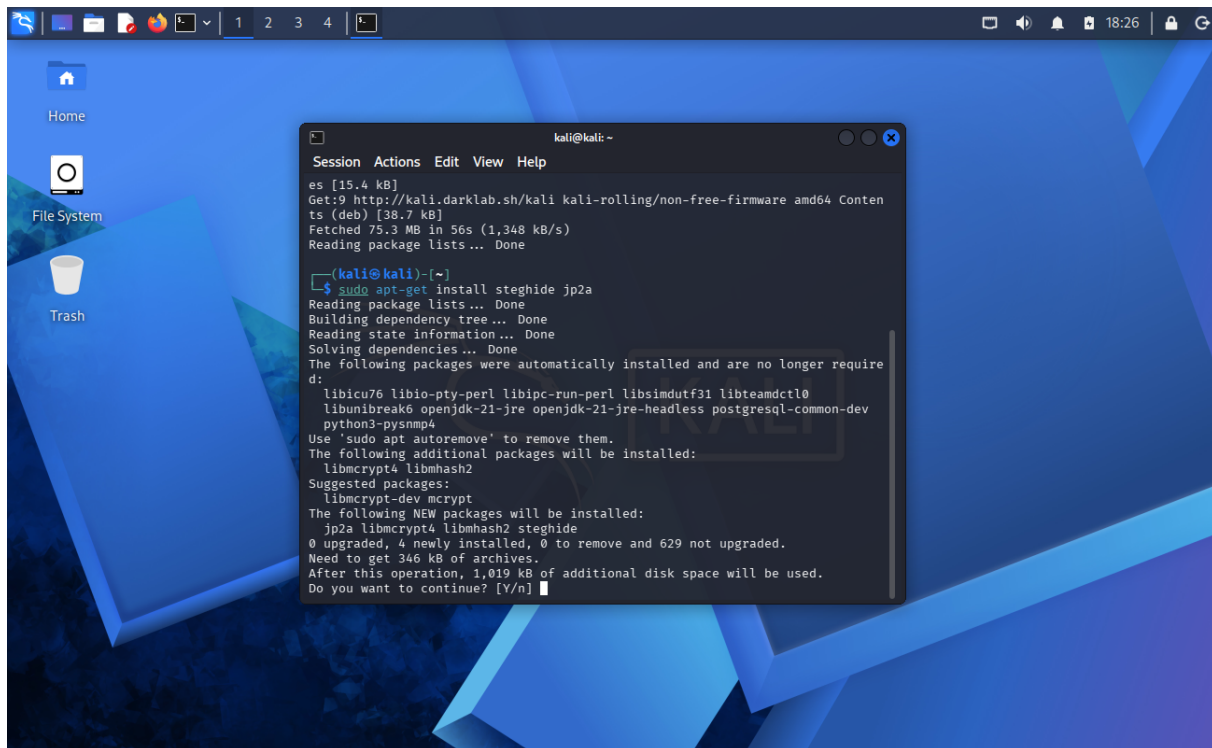


Figura 2: Instalación de `steghide` y `jp2a`.

2. Una vez instaladas las herramientas, se procedió a inspeccionar la imagen objetivo. Sabiendo que la imagen contenía datos inyectados, se ejecutó el comando de extracción de `steghide`

sobre el archivo `mao-shi.jpg`. Al solicitar la frase de contraseña, se introdujo la credencial proporcionada: **eje4**. El algoritmo descifró y extrajo exitosamente un archivo oculto, generando en nuestro directorio local un ejecutable de bash llamado `silent.sh`.

```
$ steghide extract -sf mao-shi.jpg
```

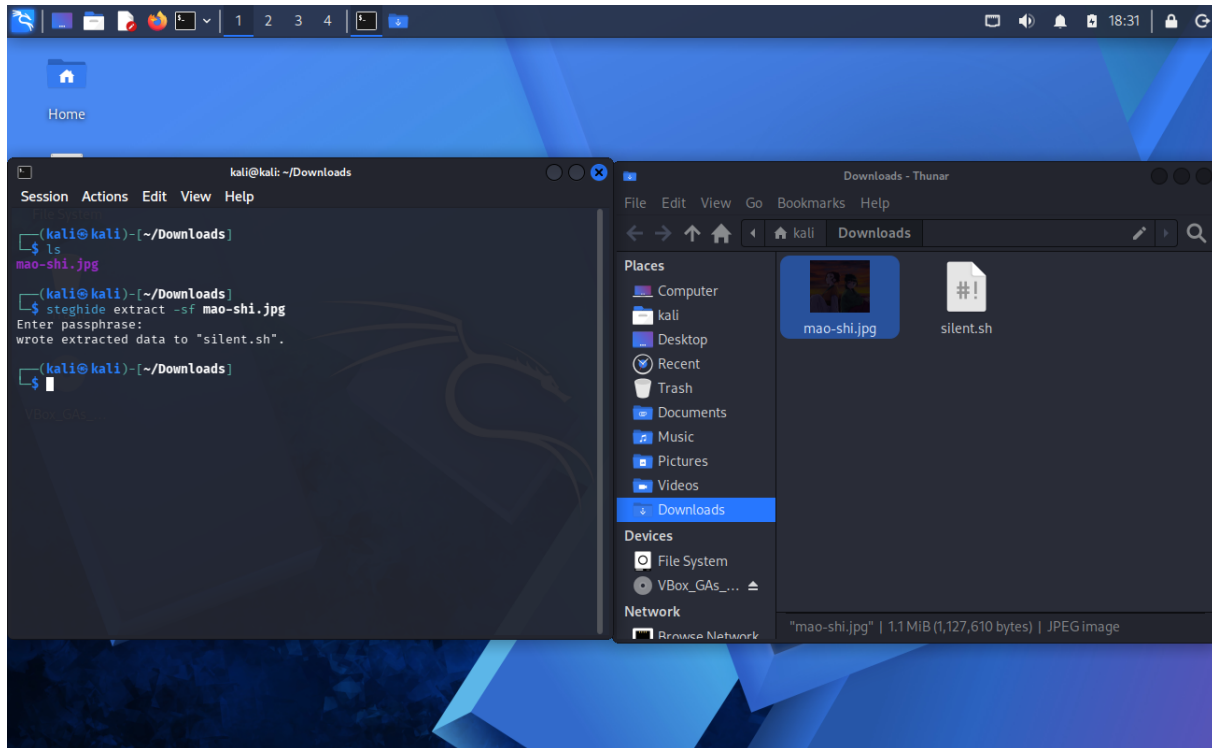


Figura 3: Extracción de datos ocultos

3. Tras otorgarle permisos de ejecución al archivo `silent.sh`, se procedió a ejecutarlo en la terminal. El resultado de este script fue la generación de un archivo `index.html`.

```
$ chmod +x silent.sh  
$ ./silent.sh
```

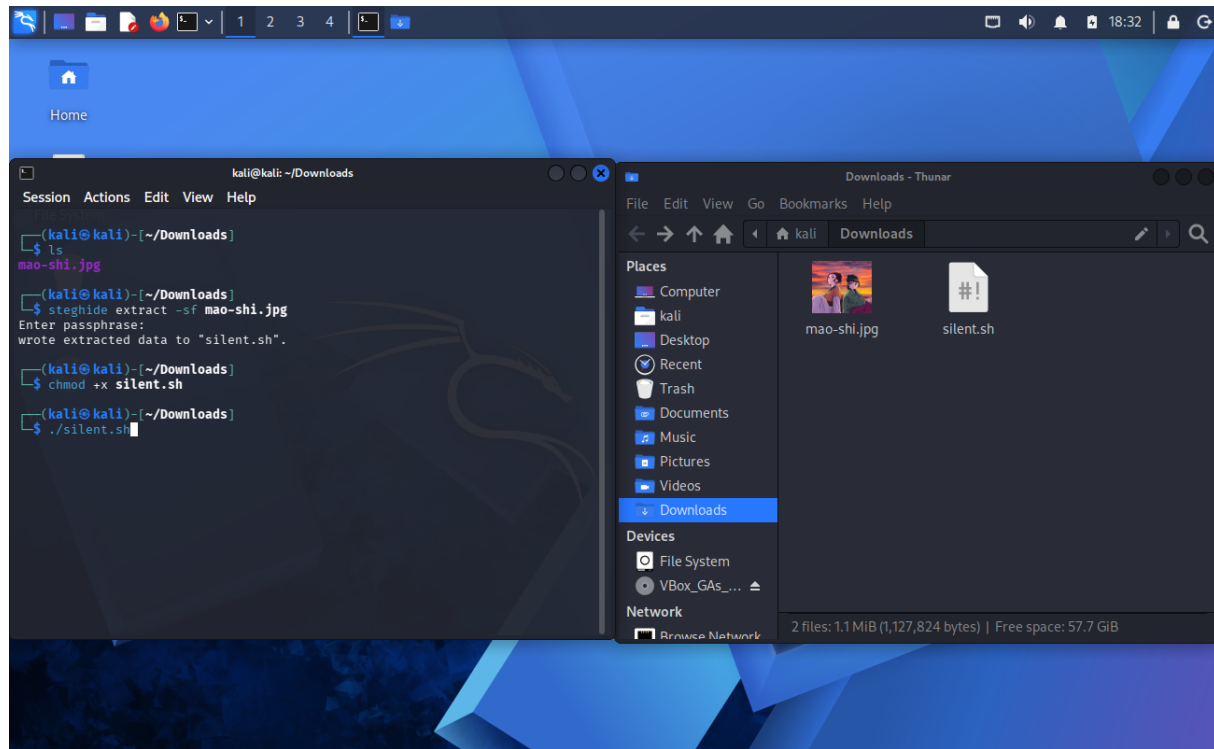



Figura 4: Permisos y ejecución de `silent.sh`.

- Al abrir el archivo `index.html` en el navegador, la pantalla principal mostraba el diseño en arte ASCII sobre un fondo negro. Sin embargo, tanto al seguir jugando con la selección de texto en el navegador, o bien, al realizar una inspección directa del código fuente del HTML, se descubrió al final del documento, el siguiente texto:

```
FLAG{s73g4n0gr4phy_15_h1dd3n}
```

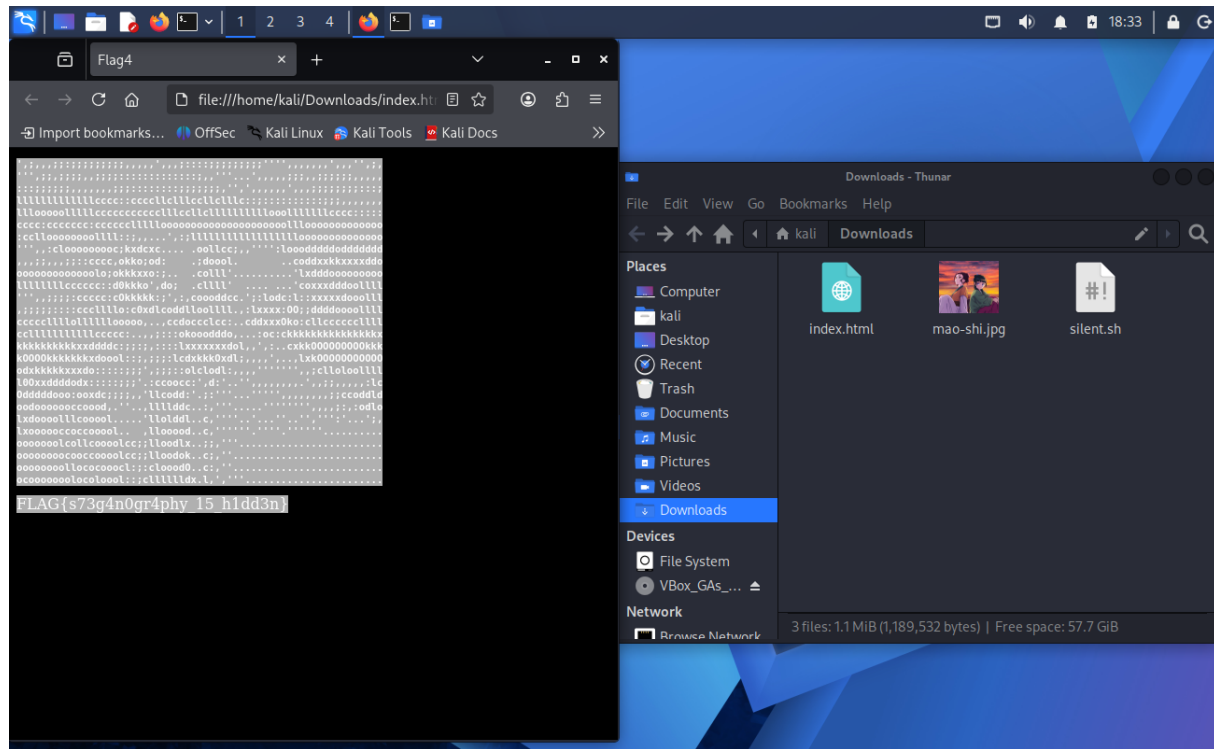


Figura 5: Inspección y descubrimiento de la bandera.

Este ejercicio demostró cómo simples archivos multimedia pueden utilizarse para cargar información o introducir scripts (.sh) en sistemas objetivo sin levantar sospechas de firewalls o antivirus, ya que a nivel de red y de sistema de archivos, el archivo original es indistinguible de una imagen JPEG normal.

La bandera recuperada fue: `FLAGs73g4n0gr4phy_15_h1dd3n`

Problema 6: Una gran tormenta se avecina

El objetivo de este problema es comprender la naturaleza y el impacto de los ataques de Denegación de Servicio (DoS) y su variante distribuida (DDoS). Específicamente, se simuló un ataque de Inundación SYN (SYN Flood) contra un servidor web. Este tipo de ataque explota el diseño Three-way Handshake del protocolo TCP, enviando una cantidad masiva de solicitudes de conexión (SYN) sin llegar a completarlas nunca (ACK), con el fin de agotar la memoria y los recursos del servidor víctima hasta dejarlo inoperante.

Para la realización de esta ejercicio, se adaptó el entorno utilizando una máquina virtual con Fedora

43 configurada en modo adaptador puente (Bridged) actuando como servidor víctima (con los servicios `httpd`, `vsftpd` y `sshd` activos), y una máquina física con Fedora 42 que actúa como atacante con la herramienta `hping3` instalada.

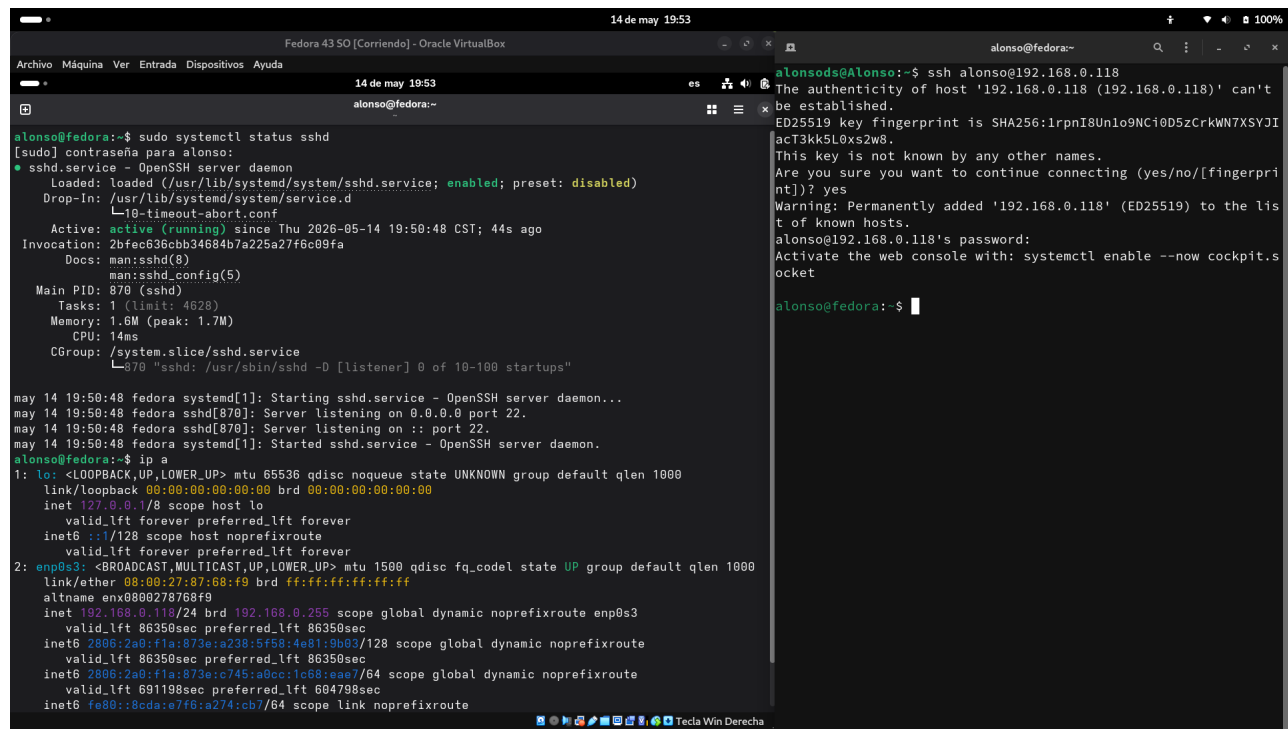


Figura 6: Prueba de conexión SSH entre la máquina virtual (víctima) y máquina física (atacante)

Con el comando `ip a`, obtenemos la IP de nuestra máquina virtual: **192.168.0.118**.

Instalamos todas las bibliotecas necesarias en nuestra máquina víctima y activamos los servicios. El repositorio brinda los comandos para Debian, por lo que los comandos en Fedora son:

```
# Instalacion de servicios en Fedora (reemplaza a apt-get)

sudo dnf update

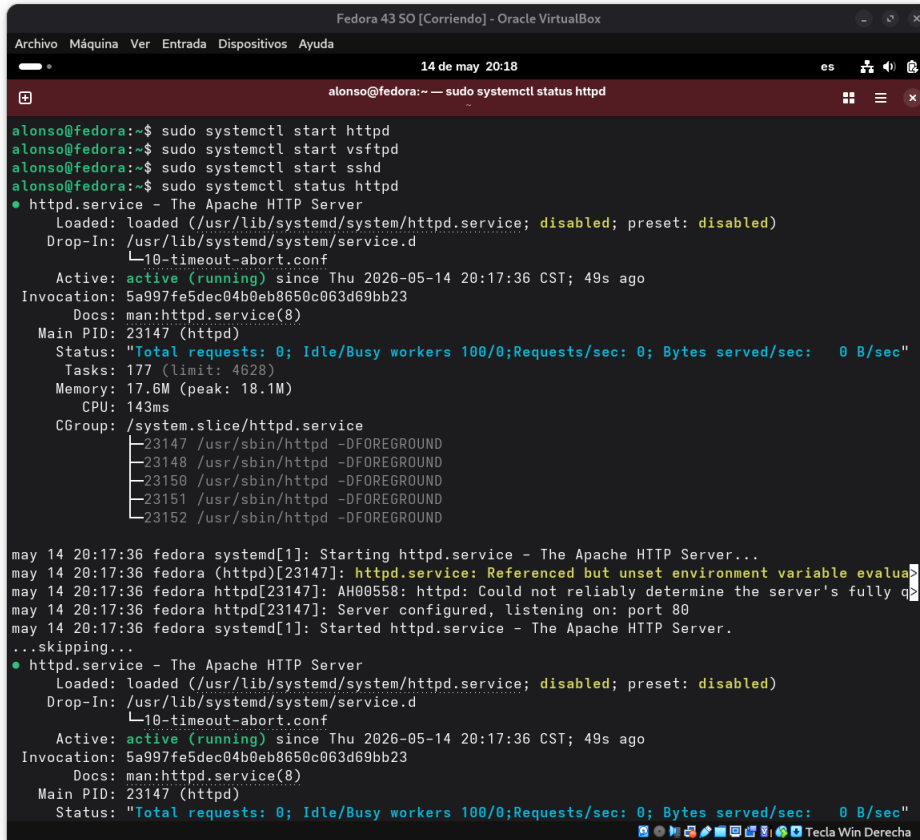
sudo dnf install httpd vsftpd openssh-server wireshark


# Iniciar los servicios (Apache se llama httpd en Fedora)

sudo systemctl start httpd

sudo systemctl start vsftpd

sudo systemctl start sshd
```



```
Fedora 43 SO [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
14 de may 20:18
alonso@fedora:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
           └─10-timeout-abort.conf
   Active: active (running) since Thu 2026-05-14 20:17:36 CST; 49s ago
   Invocation: 5a997fe5dec04b0eb8650c063d69bb23
   Docs: man:httpd.service(8)
   Main PID: 23147 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
   Tasks: 177 (limit: 4628)
   Memory: 17.6M (peak: 18.1M)
   CPU: 143ms
   CGroup: /system.slice/httpd.service
           └─23147 /usr/sbin/httpd -DFOREGROUND
             23148 /usr/sbin/httpd -DFOREGROUND
             23150 /usr/sbin/httpd -DFOREGROUND
             23151 /usr/sbin/httpd -DFOREGROUND
             23152 /usr/sbin/httpd -DFOREGROUND

may 14 20:17:36 fedora systemd[1]: Starting httpd.service - The Apache HTTP Server...
may 14 20:17:36 fedora (httpd)[23147]: httpd.service: Referenced but unset environment variable evaluat>
may 14 20:17:36 fedora httpd[23147]: AH00558: httpd: Could not reliably determine the server's fully q
may 14 20:17:36 fedora httpd[23147]: Server configured, listening on: port 80
may 14 20:17:36 fedora systemd[1]: Started httpd.service - The Apache HTTP Server.
...skipping...
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
           └─10-timeout-abort.conf
   Active: active (running) since Thu 2026-05-14 20:17:36 CST; 49s ago
   Invocation: 5a997fe5dec04b0eb8650c063d69bb23
   Docs: man:httpd.service(8)
   Main PID: 23147 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
```

Figura 7: Verificación de estatus de los servicios (en VM)

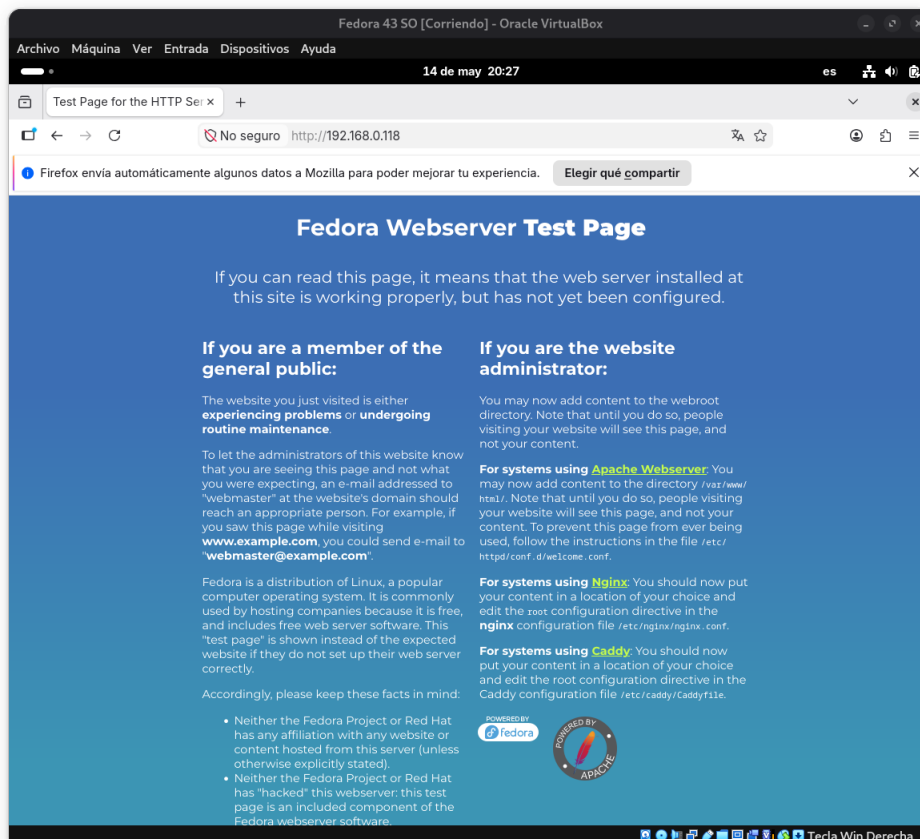


Figura 8: Comprobación del servicio Apache activo desde el navegador (en VM)

Antes de ejecutar el ataque, fue necesario confirmar que el objetivo tenía los puertos correctos abiertos. Desde la máquina atacante se ejecutó un escaneo de los primeros 1024 puertos utilizando `hping3`:

```
sudo hping3 --scan 1-1024 -S 192.168.0.118
```

Este comando envía un paquete TCP con la bandera SYN a los primeros 1024 puertos. Si el puerto está abierto, la máquina víctima responde con un SYN-ACK.

```
alonsods@Alonso:~$ sudo hping3 --scan 1-1024 -S 192.168.0.118
Scanning 192.168.0.118 (192.168.0.118), port 1-1024
1024 ports to scan, use -V to see all the replies
+-----+
|port| serv name | flags |ttl| id | win | len |
+-----+
21 ftp      : .S..A... 64   0 64240 46
22 ssh      : .S..A... 64   0 64240 46
80 http     : .S..A... 64   0 64240 46
All replies received. Done.
Not responding ports: (1 tcpmux) (2 nbp) (3 ) (4 echo) (5 rje)
(6 zip) (7 echo) (8 ) (9 discard) (10 ) (11 systat) (12 ) (13 x
aytime) (14 ) (15 netstat) (16 ) (17 qotd) (18 ) (19 chargen) (
20 ftp-data) (23 telnet) (24 lmtp) (25 smtp) (26 ) (27 nsw-fe)
(28 ) (29 msg-icp) (30 ) (31 msg-auth) (32 ) (33 dsp) (34 ) (35
) (36 ) (37 time) (38 ) (39 rlp) (40 ) (41 graphics) (42 names
erver) (43 nickname) (44 mpm-flags) (45 mpm) (46 mpm-snd) (47 ni
-ftp) (48 auditd) (49 tacacs) (50 re-mail-ck) (51 la-maint) (52
xns-time) (53 domain) (54 xns-ch) (55 isi-gl) (56 xns-auth) (5
7 ) (58 xns-mail) (59 ) (60 ) (61 ni-mail) (62 acas) (63 whois+
) (64 covia) (65 tacacs-ds) (66 sql*net) (67 bootps) (68 bootp
c) (69 tftp) (70 gopher) (71 netrjs-1) (72 netrjs-2) (73 netrjs
-3) (74 netrjs-4) (75 ) (76 deos) (77 ) (78 vettcp) (79 finger)
(81 ) (82 xfer) (83 ) (84 ctf) (85 mit-ml-dev) (86 mfcobol) (8
7 ) (88 kerberos) (89 su-mit-tg) (90 dnsix) (91 mit-dov) (92 )
(93 dcp) (94 objcall) (95 supdup) (96 dixie) (97 swift-rvf) (98
tacnews) (99 metagram) (100 newacct) (101 hostname) (102 iso-t
sap) (103 gppitnp) (104 acr-nema) (105 csnet-ns) (106 poppassd)
(107 rtelnet) (108 snagas) (109 pop2) (110 pop3) (111 sunrpc)
(112 mcidas) (113 auth) (114 ) (115 sftp) (116 ansanotify) (117
uucp-path) (118 sqlserv) (119 nntp) (120 cfdptkt) (121 erpc) (
122 smakynet) (123 ntp) (124 ansatrader) (125 locus-map) (126 n
xedit) (127 locus-con) (128 gss-xlicen) (129 pwdgen) (130 cisco
-fna) (131 cisco-tna) (132 cisco-sys) (133 statsrv) (134 ingres
-net) (135 epmap) (136 profile) (137 netbios-ns) (138 netbios-d
gm) (139 netbios-ssn) (140 emfis-data) (141 emfis-cntl) (142 bl
-idm) (143 imap) (144 ) (145 uaac) (146 iso-tp0) (147 iso-ip) (
148 jargon) (149 aed-512) (150 sql-net) (151 hems) (152 bftp) (
153 sgmp) (154 netsc-prod) (155 netsc-dev) (156 sqlsrv) (157 kn
```

Figura 9: Escaneo de puertos abiertos con `hping3`

Como observación adicional, estos mismos puertos abiertos también pueden ser escaneados con `nmap`.

```
nmap -p 1-1024 192.168.0.118
```

```
alonsods@Alonso:~  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0  
Nmap scan report for 192.168.0.118  
Host is up (3.7s latency).  
Not shown: 848 filtered tcp ports (no-response), 173 filtered t  
cp ports (host-unreach)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
80/tcp    open  http  
  
Nmap done: 1 IP address (1 host up) scanned in 158.82 seconds  
alonsods@Alonso:~$ c
```

Figura 10: Escaneo de puertos abiertos con `nmap`

Ejecutamos Wireshark en nuestra máquina virtual y luego seleccionamos la interfaz de red que se está usando.

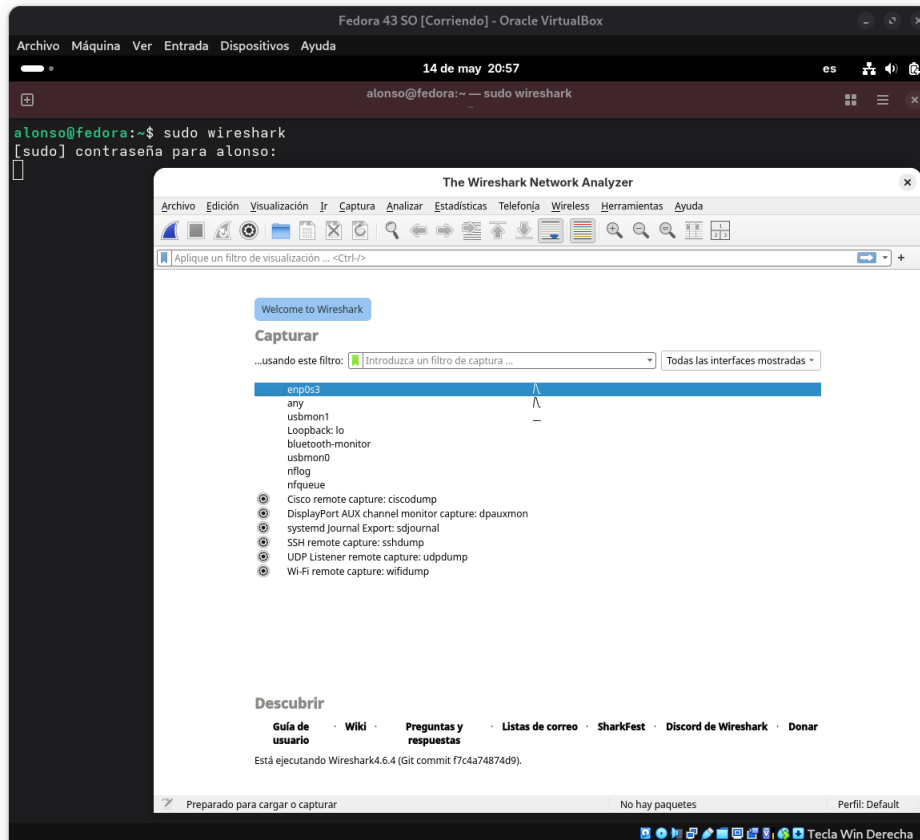


Figura 11: Ejecución de Wireshark (en VM)

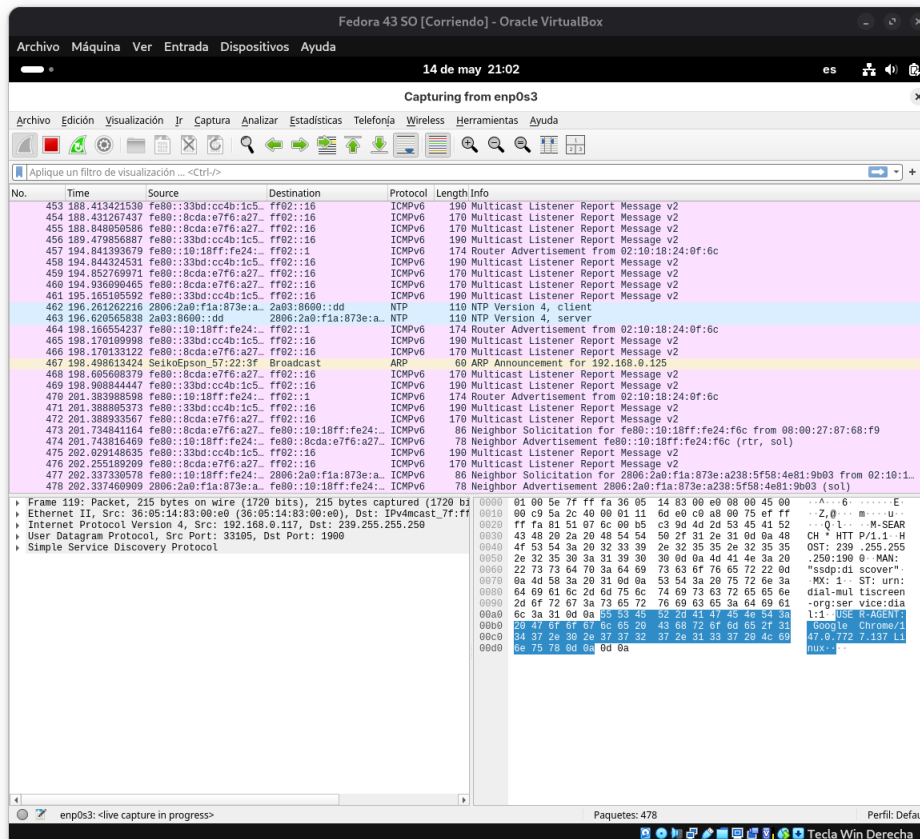


Figura 12: Se elige la interfaz de red para visualizar el tráfico de paquetes

Una vez identificado el puerto 80 abierto, se procedió a lanzar el primer ataque de inundación (flood) y se monitoreó el tráfico en la máquina víctima utilizando Wireshark.

```
sudo hping3 -S -V --flood -p 80 192.168.0.118
```

Observamos que la red se satura instantáneamente con una gran cantidad de paquetes que ingresan a rápida velocidad, en particular, este tráfico TCP va dirigido exclusivamente al puerto 80 con la bandera [SYN] activada. Todos los paquetes maliciosos provienen de la misma dirección IP (la de nuestra máquina atacante).

Podemos notar que a diferencia del tráfico legítimo, en donde existe el saludo completo: SYN (cliente) → SYN-ACK (servidor) → ACK (cliente), seguido de la transferencia de datos; en este tráfico malicioso, el servidor responde rápidamente con [SYN, ACK], pero el atacante ignora estas respuestas y jamás envía el paquete ACK final, dejando las conexiones abiertas.

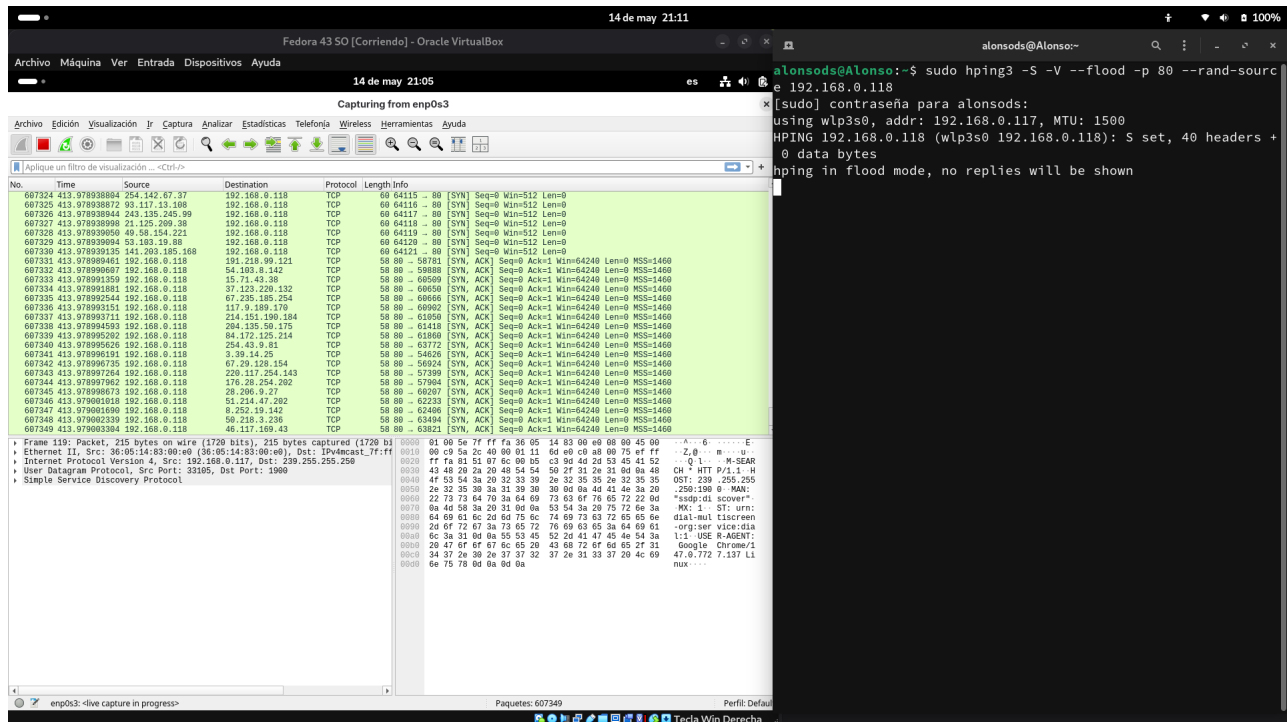


Figura 13: Ataque DDoS visto desde Wireshark

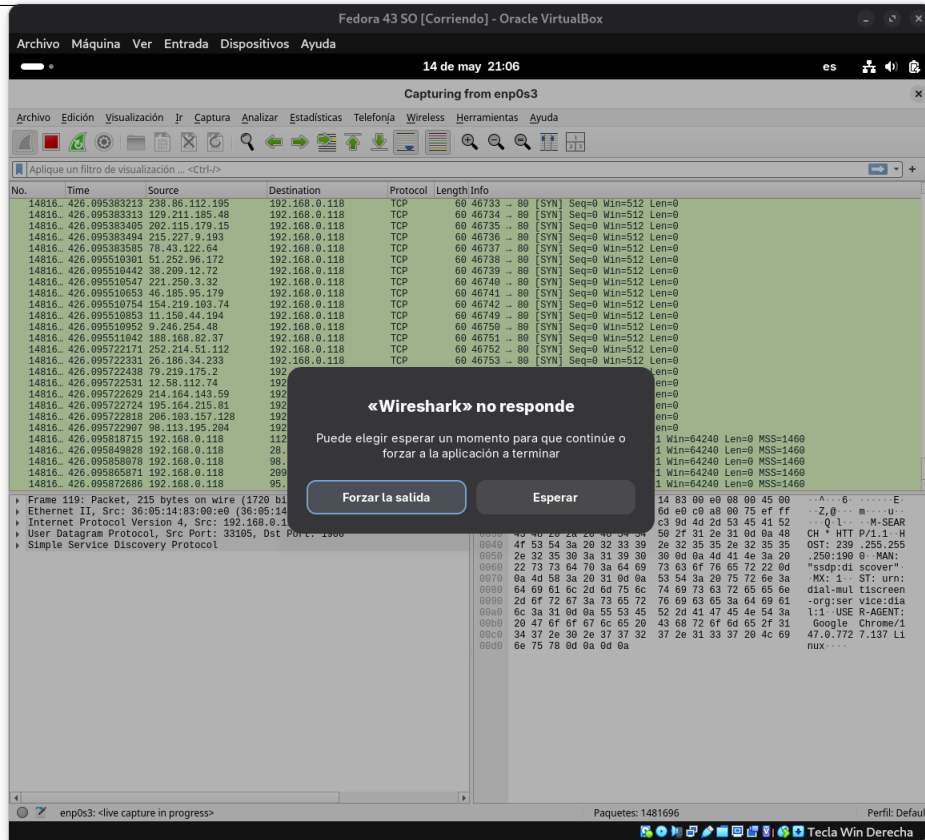


Figura 14: Crasheo en Wireshark a causa de la cantidad de tráfico de red entrante

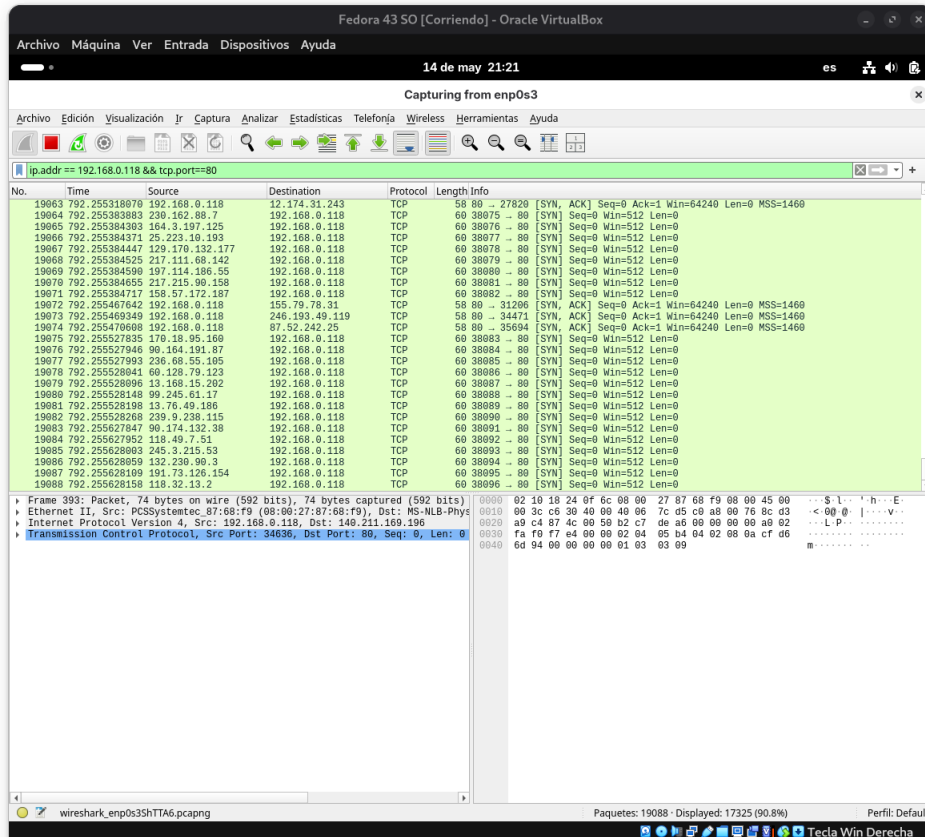


Figura 15: Nuevo ataque pero con filtro ip.addr == 192.168.0.118 && tcp.port == 80

Al aplicar el filtro de visualización en Wireshark, aislamos el tráfico dirigido al servidor web. Observamos nuevamente una alta cantidad de paquetes [SYN], pero con una diferencia en la columna de origen (Source). A diferencia del primer ataque donde la IP atacante era visible y constante, ahora cada paquete entrante proviene de una dirección IP completamente distinta.

Este ejercicio demostró la vulnerabilidad inherente de los protocolos de conexión cuando se enfrentan a volúmenes de tráfico diseñados para agotar sus recursos.

Problema 7: ¡Hey! No mires más de lo necesario.

Para este ejercicio se pudo visualizar la importancia del cifrado en las conexiones remotas, contrastando un protocolo antiguo e inseguro como Telnet, contra el estándar actual que es Secure Shell (SSH). Telnet permite el acceso en modo terminal a una máquina remota, sin embargo, carece de mecanismos de confidencialidad. Por su parte, SSH (Secure Shell) establece un canal seguro donde toda la información transmitida se encuentra cifrada. Mediante la interceptación de paquetes con Wireshark, se pondrá en práctica por qué el uso de protocolos con texto plano representa una vulnerabilidad crítica.

En esta ocasión, el servidor víctima opera en la VM Fedora 42 (en lugar de Debian) con ip **192.168.0.118**. Mientras que nuestra máquina atacante será en la VM de Kali Linux.

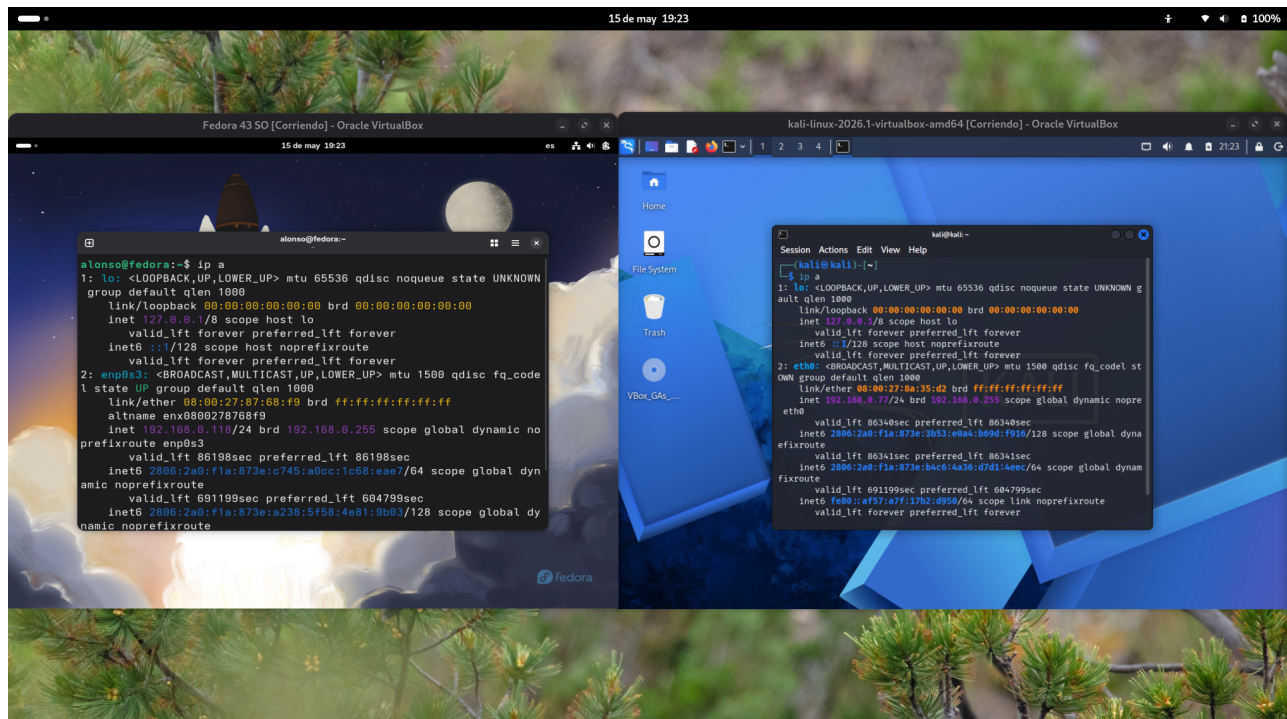


Figura 16: Máquina víctima (Fedora) y máquina atacante (Kali Linux)

Debido a que el servidor víctima funciona con Fedora 42, una distribución Linux moderna, el gestor de demonios de red heredado (xinetd) utilizado en las instrucciones originales ha sido discontinuado y eliminado de sus repositorios oficiales. Por lo tanto con ayuda Gemini, se adaptaron los comandos para Fedora para la inicialización y configuración del entorno:

Instalación de paquetes

```
sudo dnf update
sudo dnf install telnet-server telnet openssh-server
```

Inicialización de servicios

```
sudo systemctl daemon-reload
sudo systemctl enable telnet.socket
sudo systemctl restart telnet.socket
sudo systemctl enable sshd
```

Configuración de Firewall nativo

```
sudo firewall-cmd --add-service=telnet
sudo firewall-cmd --add-service=ssh
```

Preparamos Wireshark en Kali Linux para poder visualizar el intercambio de paquetes. Elegimos la interfaz de red correcta.

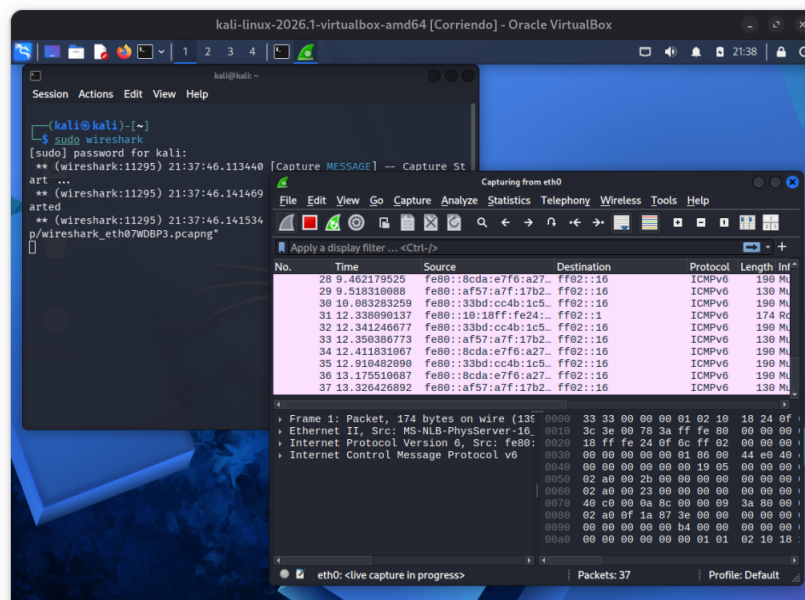


Figura 17: Wireshark en Kali Linux

Como iniciaremos con el análisis de tráfico de Telnet (en el puerto 23), aplicaremos el siguiente filtro en Wireshark: `ip.addr == 192.168.0.118 && tcp.port == 23`.

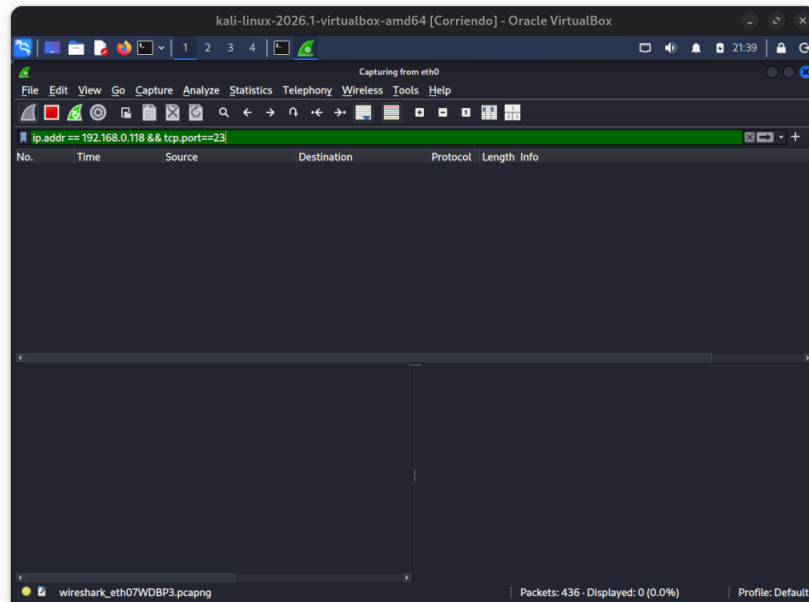


Figura 18: Filtro para Telnet en Wireshark

Con el servidor preparado, desde la máquina atacante (Kali Linux), se inició una sesión remota en otra terminal mediante el comando `telnet 192.168.0.118`, autenticándose con las credenciales del sistema y ejecutando comandos básicos como `ls` y `uname -a`. Simultáneamente, se capturaba el tráfico con Wireshark.

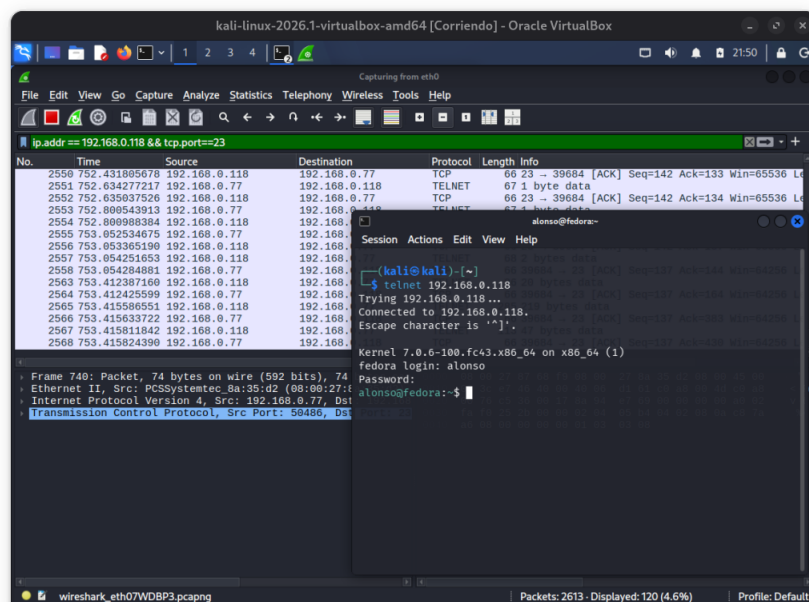


Figura 19: Conexión remota a la víctima con Telnet

Posterior a esto, seleccionamos uno de los paquetes realizados con Telnet y realizamos lo siguiente para visualizar el flujo de comunicación: *Click derecho* → *Follow* → *TCP Stream*

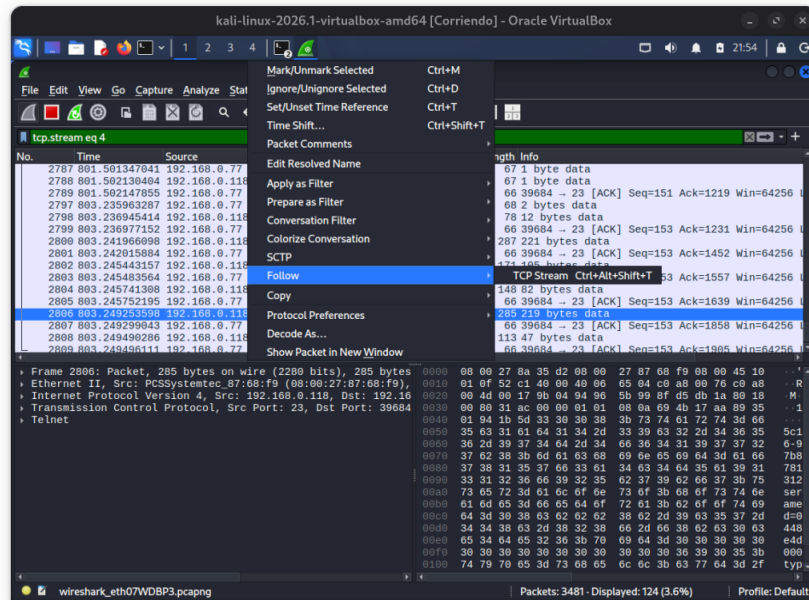


Figura 20: Pasos para la visualización del flujo TCP

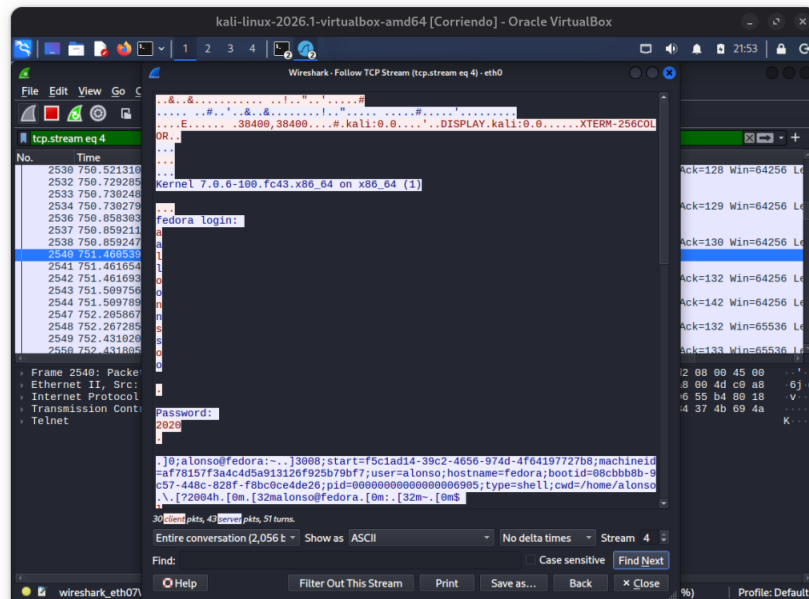


Figura 21: Visualización del usuario y contraseña en texto plano en el flujo

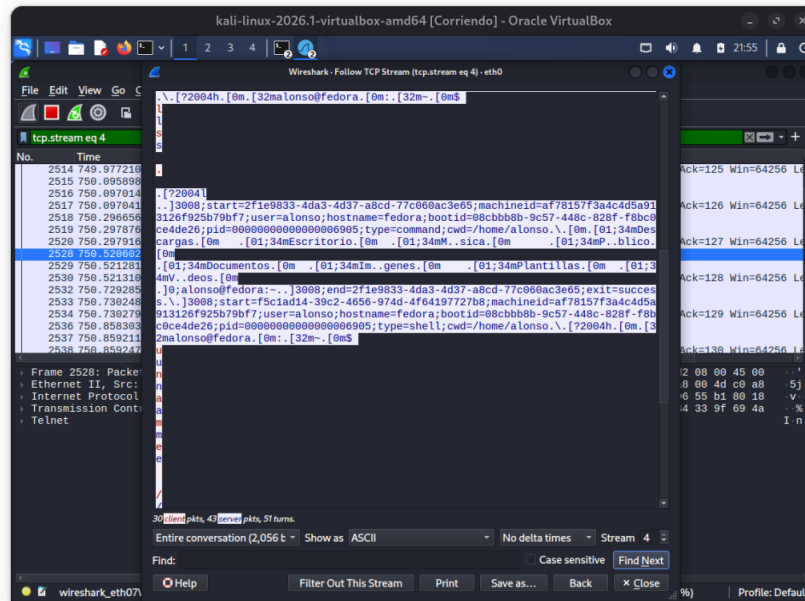


Figura 22: Visualización de comandos ejecutados en texto plano en el flujo

Al seguir el flujo TCP (TCP Stream) en Wireshark, se hace evidente la principal debilidad de Telnet: la ausencia total de cifrado. e pudo extraer absolutamente toda la interacción de la sesión en texto plano: mensajes del servidor, el nombre de usuario, la contraseña, los comandos ejecutados.

Después de esto se repite el experimento pero cambiando el protocolo a SSH (puerto 22). Por lo tanto preparamos Wireshark con un nuevo filtro: `ip.addr == 192.168.0.118 && tcp.port == 22`.

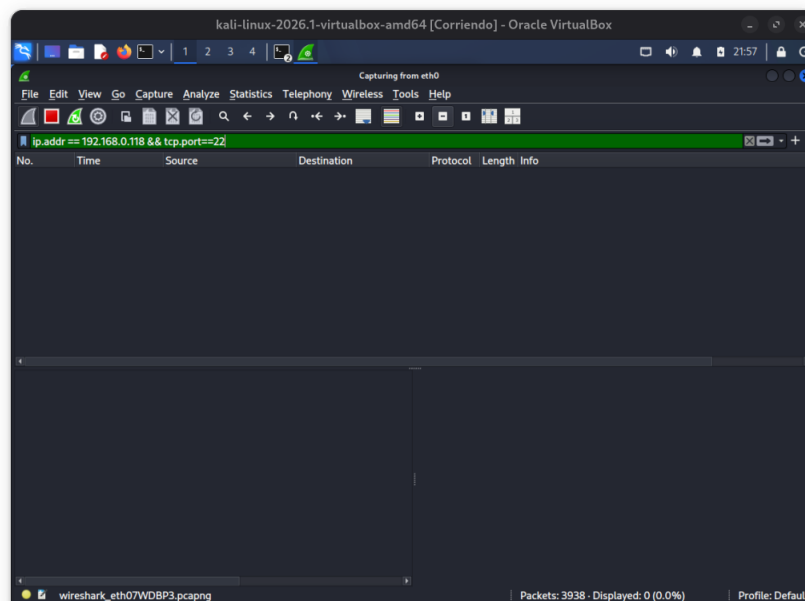


Figura 23: Filtro para SSH en Wireshark

Realizamos una nueva conexión remota usando SSH: `ssh alonso@192.168.0.118`.

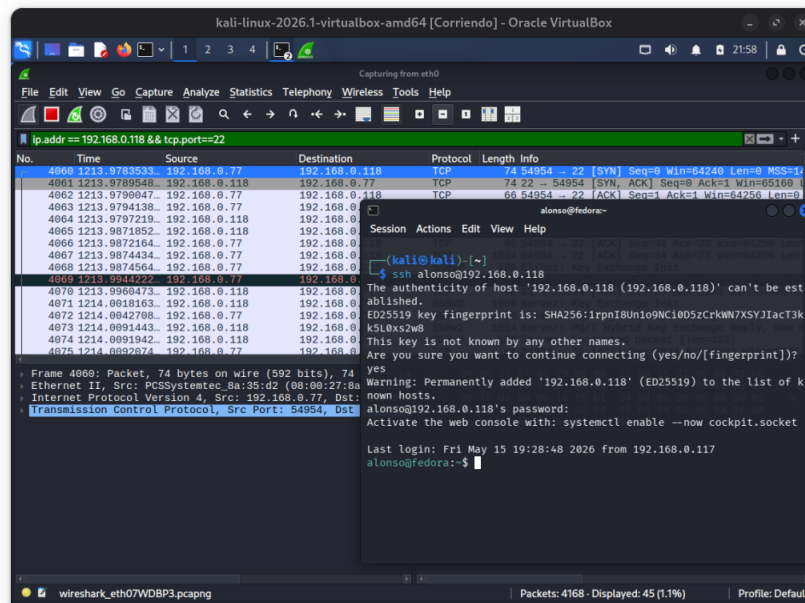


Figura 24: Conexión remota a la víctima con SSH

Posteriormente, seleccionamos cualquier paquete (ahora con protocolos SSHv2) y seguimos los mismos pasos para visualizar el flujo TCP.

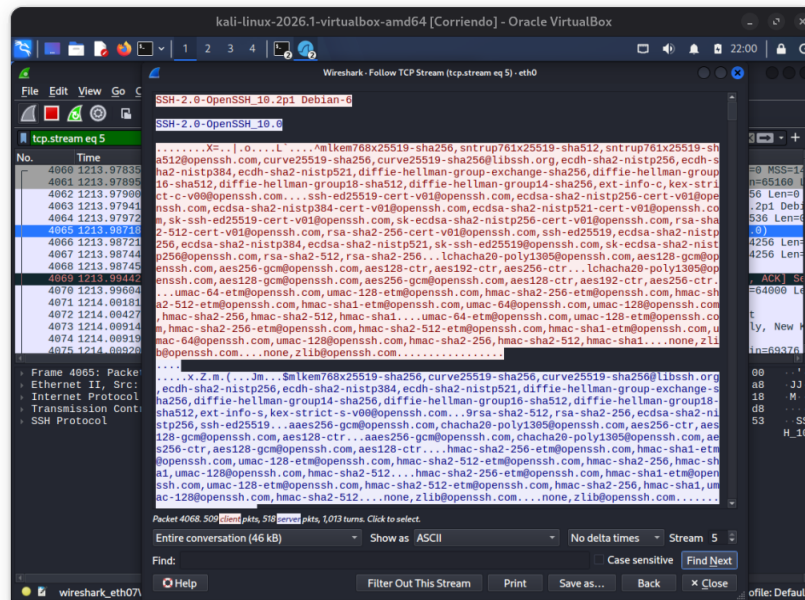


Figura 25: Visualización de comunicación cifrada en el flujo

Al abrir el flujo TCP de la captura de SSH, las primeras líneas muestran el intercambio de versiones y luego, varios algoritmos de cifrado y hasheo en texto claro. Sin embargo, después de lo anterior, todo el contenido de la pantalla se convierte en caracteres ilegibles. Es claro, que a diferencia de

Telnet, el tráfico de SSH está robustamente cifrado. Es criptográficamente inviable para un atacante extraer el nombre de usuario, la contraseña, o los comandos ejecutados durante la sesión a partir de este flujo de datos interceptado.

Podemos concluir las razones de las porque Telnet es obsoleto y está en desuso en entornos modernos. Cualquier atacante en la red local con técnicas especializadas, puede comprometer un servidor inmediatamente si se usa Telnet. SSH soluciona esta vulnerabilidad estableciendo un túnel criptográfico seguro, garantizando la confidencialidad e integridad de la conexión remota.